Acceptance Test Report

Generated 2026-10-11 from `tests/acceptance-test-report.md`

D5 deliverable. Results for the 12 mandatory acceptance tests (lab spec §7), plus the required test-evidence summary and bug log (§7.1). All automated tests run against the isolated `bcis_billing_test` database via `cd api && npm test` (Vitest + Fastify `inject`, no real HTTP socket) and `cd desktop && npm test` (Vitest, Node environment). Key desktop workflows are also driven end to end through the real Electron app with Playwright (`cd desktop && npm run e2e`, against a throwaway migrated-and-seeded database). Last full run: **246/246 API tests, 68/68 desktop tests, 4/4 end-to-end tests passing; lint 0 errors.**

Test Plan

Objective. Show that money is never lost or duplicated, that posted records are only reversed (never edited or deleted), that authorization is enforced by the server, and that three clients can work at once without corruption — the spec 7 acceptance criteria.

Levels and tools.

Level	What it covers	Tool / command	Environment
Unit / domain	Money parsing and cents math, invoice status rule, aging buckets, allocation order, permission defaults, server-address rules	Vitest (`api`, `desktop`: `npm test`)	Node
API / integration	Every route: validation (400), authentication (401), permission (403), conflicts (409), posting transactions, audit rows, concurrency via parallel requests	Vitest + Fastify `inject`	Real PostgreSQL, `bcis_billing_test` (truncated per suite)
End to end	Key desktop workflows through the real Electron UI	Playwright `_electron` (`desktop`: `npm run e2e`)	Built app + API on port 3199 + throwaway `bcis_billing_e2e` DB, migrated and seeded, dropped afterwards
Manual / visual	Layout, printing, three-PC demo	Live sweep, `docs/concurrency-demo-checklist.md`	Seeded scratch DB; screenshots in `tests/screenshots/`
Static	Types and lint	`npm run typecheck`, `npm run lint`	—

Test data. Synthetic only: per-suite fixtures in the API tests and the demo seed (`npm run db:seed`: 50 subscribers, 62 accounts, 74 invoices with 12 overdue across three aging buckets, 38 payments incl. GCash verified/rejected/pending, two batches, one reversal, a suspension and reconnection cycle). The development database is never used by tests.

Entry criteria. Migrations apply cleanly to an empty database; typecheck passes. **Exit criteria.** All 12 acceptance tests pass; all automated suites pass; lint has no errors; every defect found has a fix and a regression test or documented regression check (bug log below).

Traceability. Each AT section below names the test file and test case that proves it.

AT-01 — Exact payment

Test data: Invoice ₱999; payment ₱999. **Expected**: Remaining ₱0; invoice PAID; ledger balanced; receipt created. **Result: PASS.** `api/tests/payments.test.ts` → `describe("payment posting") > it("cash payment allocates exact to oldest invoice and marks it paid")`. Asserts the invoice status becomes `paid`, the full amount is allocated via `payment_allocations`, and a `receiptNumber` is present on the posted payment.

AT-02 — Partial payment

Test data: Invoice ₱999; payment ₱500. **Expected**: Remaining ₱499; PARTIALLY_PAID; allocation and ledger correct. **Result: PASS.** `api/tests/payments.test.ts` → `it("partial payment leaves partially_paid")`. Asserts invoice status `partially_paid` and an allocation row of exactly ₱500.00.

AT-03 — Advance payment

Test data: Monthly ₱1,000; payment ₱3,000. **Expected**: Allocation/credit follows documented policy without losing value. **Result: PASS.** `api/tests/payments.test.ts` → `it("advance payment preserves unapplied amount")`. Documented policy: oldest-first allocation consumes every open invoice in date order; any amount left over after every open invoice is fully paid is returned as `unappliedAmount` on the payment response rather than being discarded — it is available to apply the moment the next invoice is generated for that account (same oldest-first rule then applies it first). No value is ever lost or silently dropped. Also: `api/tests/search-and-preview.test.ts` → `it("reports the advance portion of an overpayment as unapplied")` (the cashier's allocation preview shows the same policy before posting), and the demo seed posts three three-month advance payments.

AT-04 — Oldest-first arrears

Test data: August ₱999 + September ₱999; payment ₱1,200. **Expected**: August ₱0; September ₱798 remaining. **Result: PASS.** `api/tests/payments.test.ts` → `it("oldest-first multi-invoice allocation")` and `it("verification allocates oldest-first across multiple invoices with exact remainder")` (GCash path). Allocation order is `ORDER BY issue_date ASC, id ASC` (`allocateToOpenInvoices` in `api/src/routes/payments.ts`), applied identically whether the payment posts immediately (cash) or on GCash verification.

AT-05 — Duplicate GCash reference

Test data: Post an already-used verified GCash reference. **Expected**: System blocks or explicitly warns per documented policy. **Result: PASS.** `api/tests/payments.test.ts` → `it("GCash payment records pending verification and rejects duplicate references")`. Documented policy:

`gcash_payment_details.gcash_reference_number` has a database unique index — a second submission with the same reference is rejected with `409 Duplicate receipt number or GCash reference` at the `POST /payments` boundary, before it can ever reach verification.

AT-06 — Payment reversal

Test data: Reverse an incorrectly posted payment. **Expected**: Original remains visible; linked reversal exists; balances restore; actor/reason audited. **Result: PASS.** `api/tests/payments.test.ts` → `describe("payment reversal") > it("reverses payment, restores invoice state, writes audit row")`. The original `payments` row is never deleted (status flips to `reversed`); a `payment_reversals` row links back to it with reason + `reversed_by_user_id`; affected invoices are recalculated from remaining posted allocations; an `audit_logs` row with action `payment.reverse` is written in the same transaction.

AT-07 — Collector balanced remittance

Test data: Cash collected ₱20,000; remitted ₱20,000. **Expected**: Difference ₱0; batch may reconcile/close. **Result: PASS.** `api/tests/collection-batches.test.ts` → `it("balanced remittance")`. A balanced batch reconciles without an acknowledgement and is then closed with explicit confirmation (`it("AT-08: … closing needs confirmation")`). Expected cash counts **cash only**: `it("non-cash collections are summarized separately and never create a shortage")`.

AT-08 — Collector shortage

Test data: Cash collected ₱20,000; remitted ₱19,500. **Expected**: ₱500 shortage displayed; cannot silently close as balanced. **Result: PASS.** `api/tests/collection-batches.test.ts` → `it("shortage")` (and `it("overage")` for the symmetric case). `shortage_amount` / `overage_amount` are stored columns, always computed and shown — never coerced to zero. `it("AT-08: a shortage cannot be reconciled without acknowledgement, and closing needs confirmation")` uses the spec's exact figures (₱20,000 collected, ₱19,500 remitted): the ₱500.00 shortage is shown, reconciling without an acknowledgement and explanation is refused (409), the shortage is still ₱500.00 after reconciliation, a Cashier cannot close (403), and closing requires `confirm: true`. Screenshot: `tests/screenshots/18-batch-reconciliation-shortage.png`.

AT-09 — Concurrent users

Test data: Two/three PCs post/read different valid operations. **Expected**: No data corruption, duplicate numbering, or unauthorized cross-session effect. **Result: PASS.** Covered per-operation, each exercising real overlapping requests via `Promise.all` /parallel `app.inject` calls against the same row:

- `api/tests/invoices.test.ts` → `it("concurrent duplicate creation yields exactly one invoice and 409s")`.
- `api/tests/payments.test.ts` → `it("two concurrent payments cannot over-allocate the same invoice")`, `it("concurrent verify requests on the same pending payment: exactly one succeeds, no duplicate allocation")`, `it("concurrent verify and reject race on the same pending payment: exactly one succeeds, end state is consistent")`.
- `api/tests/collection-batches.test.ts` → `it("concurrent remit safety: only one succeeds")`.

- `api/tests/receivables.test.ts` → `it("concurrent suspend: one 200 and one 409, status suspended once")`.

Manual three-client procedure for the live demo is documented in `docs/deployment-guide.md` §5.

AT-10 — Authorization

Test data: Cashier attempts admin-only user/backup operation. **Expected**: Server rejects operation even if the API is called directly. **Result: PASS.** `api/tests/backup.test.ts` → `it("AT-10 style: a non-Owner role is rejected by the server even though it never sees the button")` (explicitly named for this test). Every route file additionally has its own 401 (unauthenticated) and 403 (wrong role) tests — e.g. `api/tests/users.test.ts` → `it("Administrator (non-Owner) cannot create a user")`, `it("Cashier cannot replace roles")`. These call the route directly via `app.inject`, bypassing any UI, which is exactly the "even if the API is called directly" condition. Since the move to granular permissions, `api/tests/permissions.test.ts` also shows that revoking `payment.reverse` from a role takes effect on the next request (403 naming the missing permission) and that only `user.manage` can edit the matrix. End to end: `desktop/e2e/workflows.e2e.ts` → `"cashier cannot reach owner-only areas, and the server refuses them even when called directly"` calls the backup API through the running app's own bridge as a Cashier and gets a refusal.

AT-11 — Duplicate billing

Test data: Run billing generation twice for the same period. **Expected**: No duplicate finalized invoice for the same service account/period. **Result: PASS.** `api/tests/invoices.test.ts` → `it("duplicate invoice for same account+period -> 409")` and `it("concurrent duplicate creation yields exactly one invoice and 409s")`. Enforced at two layers: the `invoices_service_account_period_unique` database constraint, and an in-transaction re-check before insert (so the concurrent case returns a clean `409`, not a raw constraint-violation error). The monthly billing run itself: `api/tests/invoice-workflows.test.ts` → `it("bills each active account once; a second run creates nothing for it")` (`POST /billing-periods/:id/generate` twice → one invoice per account).

AT-12 — Backup/restore

Test data: Create test backup, change data, restore approved backup. **Expected**: Restored database passes integrity check and expected records return. **Result: PASS.** `api/tests/backup.test.ts` → `describe("AT-12: full dump -> mutate -> restore integrity cycle") > it("restores a database to its backed-up state, losing only post-backup changes")`. Run against a disposable scratch PostgreSQL database (created via a raw `pg.Client` against the `postgres` maintenance database, dropped after the test) specifically so this destructive scenario never touches the shared `bcis_billing_test` that the other 12 test files depend on.

Summary

ID	Scenario	Result
AT-01	Exact payment	PASS
AT-02	Partial payment	PASS

ID	Scenario	Result
AT-03	Advance payment	PASS
AT-04	Oldest-first arrears	PASS
AT-05	Duplicate GCash reference	PASS
AT-06	Payment reversal	PASS
AT-07	Collector balanced remittance	PASS
AT-08	Collector shortage	PASS
AT-09	Concurrent users	PASS
AT-10	Authorization	PASS
AT-11	Duplicate billing	PASS
AT-12	Backup/restore	PASS

Required Test Evidence

- **Automated test output for domain/business rules** — `cd api && npm test` (246 tests in 20 files): money math, allocation, invoice status rule and states (draft/void/credited/overdue), billing runs, reversals, GCash, proofs, batches, receivables, reports, permissions, lockout, backup.
- **API/integration results for posting flows** — the same suite runs through Fastify `inject` against real PostgreSQL transactions (no mocks).
- **E2E for key desktop workflows** — `cd desktop && npm run e2e` (Playwright driving the built Electron app; 4 tests: sign-in and dashboard, Receive Payment end to end, GCash verification, role restriction with a direct API call refused). Desktop unit tests: `cd desktop && npm test` (68).
- **Screenshots / report files showing expected balances** — `tests/screenshots/` (index in its README; 10–23 come from a live sweep against a freshly migrated and seeded database) and `reports-samples/` (18 PDF/XLSX files exported from the running system, e.g. the period Statement of Account shows opening 1,999.00 + charges 1,999.00 − payments 2,500.00 = closing 1,498.00).
- **Lint** — `npm run lint` in both packages: 0 errors.
- **Bug log** — below.

Bug Log

Bug 1 — "Rendered fewer hooks than expected" crash in Service Account list

- **Discovered**: while adding the ledger/Statement-of-Account "View" navigation to `desktop/src/renderer/src/components/service-accounts/ServiceAccountList.tsx`.
- **Symptom**: clicking a row's new "View" button crashed the renderer with React's `"Rendered fewer hooks than expected"` error. Not caught by TypeScript or ESLint under this project's configuration.
- **Root cause**: three new conditional early-`return` statements (`viewingAccountId` / `ledgerAccountId` / `soaAccountId`) were placed immediately after the component's `useQuery` call, but *before* `const handleSearch = React.useCallback(...)`, which is declared further down in the same component. On the branch that took an early return, `handleSearch`'s `useCallback` never ran — violating the Rules of Hooks (every hook must run unconditionally, in the same order, on every render).

- **Diagnosis method**: root-caused with a temporary `mainWindow.webContents.on("console-message", ...)` listener forwarded to the terminal, reproducing the crash live and reading the exact React error and component stack, rather than guessing from the symptom.
- **Fix**: moved all three early-return blocks to immediately after the last hook call in the component (`handleSearch`'s declaration), with a comment recording the constraint so it isn't reintroduced: `// All hooks above must run unconditionally on every render (Rules of Hooks) — these navigation early-returns come after the last one.`
- **Regression test**: this specific failure mode (hook-order violation) is not mechanically testable by Vitest in a Node environment without a rendering harness; the standing practice adopted afterward is to grep every new/modified component for an early `return` placed before a hook declaration as part of review, and to live-click every new navigation path before marking a UI change done (per `.claude/skills/verify-desktop-ui/SKILL.md`). Verified fixed by re-running the exact click sequence that originally crashed and confirming the Ledger/Statement/Detail views now render.

Bug 2 — Installed clients could not reach a LAN server

- **Discovered**: compliance review against spec 14 ("all three client PCs can connect").
- **Root cause**: the packaged app never loads `desktop/.env`, and the main process rejected plain `http://` for every host except localhost — so the documented `http://<server-lan-ip>:3000` threw at start-up.
- **Fix**: server address is a runtime setting on the login screen, health-checked before saving; plain HTTP allowed only for loopback and private LAN ranges (`desktop/src/main/server-url.ts`).
- **Regression test**: `desktop/tests/server-url.test.ts`.

Bug 3 — GCash in a collection batch produced a false cash shortage

- **Discovered**: reviewing the batch remittance logic against spec 3.8/4.3.
- **Root cause**: expected remittance summed every posted payment in the batch, including GCash, which never passes through the collector.
- **Fix**: expected cash counts cash only; non-cash is summarized separately.
- **Regression test**: `collection-batches.test.ts` → "non-cash collections are summarized separately and never create a shortage".

Bug 4 — New service accounts could not be created from the desktop

- **Discovered**: compliance review.
- **Root cause**: `ServiceAccountList` passed empty arrays for subscribers, plans, areas and collectors to the create form, so every dropdown was empty; editing from the list also started from a partial row.
- **Fix**: the form loads its own lookups with a subscriber type-ahead; edit loads the full record first.
- **Regression check**: live UI sweep (`tests/screenshots/13`, `22`) and the API create/update tests in `service-control.test.ts`.

Bug 5 — Invoice creation and detail screens were stubs

- **Discovered**: compliance review.
- **Root cause**: "Add Invoice" and "View" on the invoice list were `alert("not yet implemented")`; the existing form also sent empty date strings, which the API rejects.
- **Fix**: working invoice form (blank optional fields omitted), invoice detail with finalize/adjust/void/print.

- **Regression test**: `invoice-workflows.test.ts`; `billing-workflows.test.ts`.

Bug 6 — Reconnection fees were recorded but never billed

- **Discovered**: compliance review of spec 3.10.
- **Root cause**: `reconnection_records.fee` had no path into billing.
- **Fix**: the next invoice picks up completed, unbilled reconnection fees once (`billed_invoice_id`).
- **Regression test**: `service-control.test.ts` → "request → technician completes → fee billed on the next invoice".

Bug 7 — Exports silently truncated at 500 rows

- **Discovered**: reviewing reports against the spec 4.5 data volumes.
- **Root cause**: `.limit(500)` in the export queries with no notice.
- **Fix**: 50,000-row cap with the truncation stated in the report subtitle.
- **Regression test**: `management-reports.test.ts` (exports), plus report totals checked to the cent.

Bug 8 — Pages opened at the previous page's scroll position

- **Discovered**: live UI sweep with the Electron app.
- **Root cause**: the document, not the content pane, scrolls; nothing reset it on navigation.
- **Fix**: `Layout` resets the document and pane on every route change.
- **Regression check**: live sweep re-run (scroll, navigate, screenshot).